

Personal MQTT Broker Setup & Supabase Cloud Integration Guide

Production Deployment Manual for ESP32 Hardware Data Logger, Dual-Port Mosquitto, & 24/7 Bridge

DOCUMENT VERSION: 3.0 | STATUS: Production Verified | HARDWARE: ESP32-WROOM-32 / RS485 Modbus

TARGET OS: Ubuntu 22.04/24.04 LTS VPS | DATABASE: Supabase PostgreSQL | STORAGE: Supabase S3

Executive Summary

This manual provides the complete, end-to-end implementation for deploying a private, dedicated MQTT broker system for the ONE EARTH Solar (OES) monitoring ecosystem. By hosting your own broker, your solar hardware loggers establish private, authenticated telemetry communication without relying on third-party public testing servers.

SYSTEM OBJECTIVE

CORE PURPOSE: This architecture enables seamless 24/7 solar plant monitoring. The ESP32 hardware transmits real-time Modbus telemetry to your private Mosquitto server, an automated server-side bridge streams readings into Supabase tables and storage buckets, and your React Web Dashboard delivers real-time live monitoring to clients.

Table of Contents

Section	Title	Key Topics Covered	Page
1	System Architecture & Data Flow	Hardware, Broker, 24/7 Bridge, Supabase Cloud & Web Dashboard	Page 2
2	Topic Hierarchy & Device ID Scheme	Namespace format, LWT mechanism, 16 PV string payloads	Page 2
3	Step 1: Mosquitto Broker Setup	Installation, dual-listener (TCP 1883 & WS 8083), password security	Page 3
4	Step 2: 24/7 Supabase Ingestion Bridge	Node.js daemon, PM2 service, live tables, 10-min storage slots	Page 4
5	Step 3: ESP32 Hardware Configuration	Firmware Config.h setup & SoftAP wireless portal configuration	Page 5
6	Step 4: React Web Dashboard Connection	WebSocket configuration (ws:// & wss://), mixed-content handling	Page 5
7	Step 5: Testing, Diagnostics & Troubleshooting	mosquitto_sub/pub, PM2 monitoring, common error resolution matrix	Page 6

1. COMPLETE SYSTEM ARCHITECTURE & DATA FLOW

SECTION 1

The OES IoT ecosystem operates on a decoupled multi-tier architecture where hardware data collection, cloud message routing, persistent database storage, and real-time frontend visualization function together in perfect harmony:

System Architecture Diagram



2. TOPIC HIERARCHY & DEVICE ID SCHEME

SECTION 2

Communication is strictly organized using an isolated namespace pattern to prevent interference between different sites and devices:

- **oes/<DEVICE_ID>/telemetry:** High-frequency telemetry stream published every 5 seconds. Contains instantaneous active power (ac_w), energy yield (e_day, e_tot), grid voltage, grid current, and 16 individual PV string power metrics.
- **oes/<DEVICE_ID>/status:** Device availability topic with Last Will & Testament (LWT). Published with {"online": true} on startup, and automatically broadcast by the broker with {"online": false} if the hardware disconnects abruptly.
- **oes/<DEVICE_ID>/heartbeat:** Low-frequency health ping published every 30 seconds. Reports uptime, Wi-Fi RSSI signal strength, and free ESP32 heap memory.
- **oes/<DEVICE_ID>/cmd:** Remote command channel subscribed by the ESP32. Enables the cloud dashboard to remotely reboot the logger, change Modbus polling intervals, or sync inverter profiles.

3. STEP 1: INSTALLING & CONFIGURING MOSQUITTO

STEP 1

Execute the following steps on your Ubuntu 22.04 or 24.04 LTS server (DigitalOcean, AWS EC2, Hetzner, or private Linux server):

1. Update Packages & Install Mosquitto

Terminal Commands

```
# Update server packages
sudo apt update && sudo apt upgrade -y

# Install Mosquitto broker and CLI client utilities
sudo apt install -y mosquitto mosquitto-clients

# Ensure service is enabled to launch on system boot
sudo systemctl enable mosquitto
```

2. Configure Multi-Port Listeners (TCP 1883 + WebSockets 8083)

Create the dedicated configuration file at `/etc/mosquitto/conf.d/oes.conf`:

Broker Configuration (`/etc/mosquitto/conf.d/oes.conf`)

```
# /etc/mosquitto/conf.d/oes.conf
# =====
# ONE EARTH Solar - Dedicated Mosquitto Broker Config
# =====

# Persistence settings
persistence true
persistence_location /var/lib/mosquitto/

# Logging configuration
log_dest file /var/log/mosquitto/mosquitto.log
log_type error
log_type warning
log_type notice
log_type information

# -----
# 1. Hardware Listener for ESP32 Data Loggers (Raw TCP)
# -----
listener 1883 0.0.0.0
protocol mqtt

# -----
# 2. WebSocket Listener for React Web Dashboard & Mobile
# -----
listener 8083 0.0.0.0
protocol websockets

# -----
# 3. Security & Access Control
# -----
allow_anonymous false
password_file /etc/mosquitto/passwd
```

3. Create Authenticated User Credentials

Authentication Setup

```
# Create user 'oes_admin' with an encrypted password
sudo mosquitto_passwd -c /etc/mosquitto/passwd oes_admin
# Enter your secure password (e.g. SolarAdmin#2026)

# Restart Mosquitto to load the new listeners and credentials
sudo systemctl restart mosquitto

# Verify both listeners are active
sudo ss -tulpn | grep -E '1883|8083'
```

4. Configure Firewall (UFW)

UFW Firewall Rules

```
sudo ufw allow 1883/tcp    # ESP32 Hardware MQTT Port
sudo ufw allow 8083/tcp    # Web Dashboard WebSocket Port
sudo ufw allow 22/tcp      # SSH Administration Port
sudo ufw enable
```

4. STEP 2: DEPLOYING THE 24/7 SUPABASE BRIDGE

STEP 2

The bridge daemon (mqtt_supabase_bridge.cjs) runs 24/7 on the server to automatically ingest live telemetry, save 10-minute historical snapshots, upload files to Supabase Storage, and manage the offline zero-reading watchdog.

1. Install Node.js 20 & PM2 Process Manager

Node.js & PM2 Installation

```
# Install Node.js 20 LTS from NodeSource
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# Install PM2 globally for production process management
sudo npm install -g pm2
```

2. Setup Application Directory & Environment

Project Setup

```
# Create application directory
mkdir -p /home/ubuntu/oes-bridge && cd /home/ubuntu/oes-bridge

# Create package.json
cat << 'EOF' > package.json
{
  "name": "oes-mqtt-supabase-bridge",
  "version": "3.0.0",
  "main": "mqtt_supabase_bridge.cjs",
  "dependencies": {
    "@supabase/supabase-js": "^2.45.0",
    "mqtt": "^5.10.0"
  }
}
EOF

# Install dependencies
npm install
```

3. Create Environment File (.env)

Environment Configuration (.env)

```
cat << 'EOF' > .env
# Supabase Production Credentials
VITE_SUPABASE_URL=https://rldvpvjpvaoabyklttixb.supabase.co
VITE_SUPABASE_ANON_KEY=sb_publishable_QQ05J5h3_Rh16l2vPl93eA_MWVvWRAY

# Local Mosquitto Broker Connection
MQTT_BROKER=mqtt://127.0.0.1:1883
MQTT_USER=oes_admin
MQTT_PASS=SolarAdmin#2026
MQTT_PREFIX=oes
EOF
```

4. Launch Bridge Daemon with PM2 Auto-Restart

Copy your mqtt_supabase_bridge.cjs file into /home/ubuntu/oes-bridge/ and launch:

PM2 Execution Commands

```
# Start bridge process under PM2
pm2 start mqtt_supabase_bridge.cjs --name "oes-bridge"

# Save process list and configure auto-start on server reboot
pm2 save
pm2 startup
```

SUPABASE SQL FIX

DATABASE TRIGGER REQUIREMENT: If you previously dropped data_7day_history, remember to run:
DROP TRIGGER IF EXISTS trg_sync_telemetry_to_7day_history ON public.data_telemetry;
in your Supabase SQL Editor so 10-minute historical rows write to data_telemetry without error.

5. STEP 3: CONFIGURING THE ESP32 HARDWARE LOGGER

STEP 3

Configure your physical ESP32 data logger to point to your new private server using either method:

Method A: Compile-Time Configuration (Config.h)

Config.h Parameters

```
// Config.h – Hardware Settings
#define MQTT_BROKER      "YOUR_SERVER_IP"      // e.g. "159.65.120.45"
#define MQTT_PORT        1883                  // Raw TCP Port
#define MQTT_CLIENT_ID   "OES-DL-92092"
#define MQTT_USER         "oes_admin"
#define MQTT_PASS         "SolarAdmin#2026"
#define MQTT_PREFIX       "oes"
```

Method B: Wireless Configuration via SoftAP Web Portal

- **1. Enter Setup Mode:** If the ESP32 has no saved Wi-Fi credentials, it broadcasts a Wi-Fi hotspot named "OES-LOGGER-xxxx".
- **2. Open Setup Page:** Connect your phone or laptop to this Wi-Fi network and browse to <http://192.168.4.1>.
- **3. Input Server Credentials:** Enter your Server IP, Port 1883, Username "oes_admin", Password "SolarAdmin#2026", and Topic Prefix "oes".
- **4. Save & Restart:** Click Save & Connect. The ESP32 writes configuration into non-volatile flash memory (Preferences API) and connects.

6. STEP 4: CONNECTING THE REACT WEB DASHBOARD

STEP 4

The React Web Dashboard connects via WebSockets directly to port 8083 of your broker:

Web Dashboard WebSocket Setting

```
// In browser developer console or local storage settings:
localStorage.setItem('oes_mqtt_host', 'ws://YOUR_SERVER_IP:8083');
localStorage.setItem('oes_mqtt_prefix', 'oes');
```

SSL / PRODUCTION NOTE

HTTPS / WSS PRODUCTION REQUIREMENT: If your web dashboard is served over HTTPS (e.g. Vercel, Netlify, or custom domain with SSL), modern web browsers will block plain ws:// connections due to Mixed Content security rules. In production, configure an SSL certificate via Certbot and route WebSocket traffic through Nginx (wss://yourdomain.com:8083).

7. STEP 5: VERIFICATION & CLI DIAGNOSTIC TESTING

STEP 5

Run these diagnostic tests on your server terminal to confirm end-to-end functionality:

1. Monitor Live MQTT Traffic (CLI)

CLI Traffic Monitor

```
# Listen to all messages across all devices
mosquitto_sub -h 127.0.0.1 -p 1883 -u "oes_admin" -P "SolarAdmin#2026" -t "oes/#" -v
```

2. Publish a Simulated Telemetry Packet

CLI Test Publisher

```
mosquitto_pub -h 127.0.0.1 -p 1883 -u "oes_admin" -P "SolarAdmin#2026" \
-t "oes/OES-DL-92092/telemetry" \
-m '{"device_id":"OES-DL-92092","status":2,"ac_w":64800,"e_day":52.1,"pv_v":610,"ac_v":230}'
```

3. Check PM2 Ingestion Logs

PM2 Log Command

```
# View real-time logs from the bridge daemon
pm2 logs oes-bridge --lines 30
```

8. DIAGNOSTICS & TROUBLESHOOTING MATRIX

FAQ

Symptom	Root Cause	Resolution Steps
ESP32 rc=-2 (Connection Refused)	Port 1883 blocked by UFW firewall or wrong server IP address	Run "sudo ufw allow 1883/tcp". Confirm server public IP address.
ESP32 rc=5 (Not Authorized)	Mismatch in MQTT username or password	Re-run "sudo mosquitto_passwd" and verify password in Config.h.
Web Dashboard shows "Connecting..."	Port 8083 blocked or Mixed Content on HTTPS	Ensure port 8083 is open in UFW. Use ws:// for HTTP and wss:// for HTTPS.
Table data_telemetry row not saving	Orphaned trigger trg_sync_telemetry_to_7day_history active in Supabase	Run "DROP TRIGGER IF EXISTS trg_sync_telemetry_to_7day_history ON public.data_telemetry;" in SQL Editor.
Device shows OFFLINE with 0 readings	Watchdog triggered (>45 seconds without packet)	Check ESP32 Wi-Fi connection and verify RS485 A & B wiring on inverter.